DESENVOLVIMENTO DE API BACK-END · ENGENHARIA DE SOFTWARE · 6º PERÍODO

Semana 05
Contrato da API com OpenAPI

Apostila de estudo: teoria, exemplos, construção com IA e exercícios

Na Semana 04 a API ganhou banco, DTOs e um formato único de erro. Falta dizer isso a quem vai usá-la. Nesta semana o contrato da API vira um documento legível por pessoas e por máquinas: a especificação OpenAPI. Você vai entender o que um contrato fixa, como ler e escrever um documento OpenAPI, como evoluí-lo sem quebrar clientes e como o Springdoc gera esse documento a partir do código do ServiceHub. Leia antes ou depois da aula; os exercícios do fim ajudam a conferir o que ficou.

Conteúdo

1. Objetivos da semana
2. O que é um contrato de API
3. Design-first e code-first
4. OpenAPI, Swagger e Springdoc
5. A estrutura de um documento
6. JSON Schema: descrevendo dados
7. Reuso com $ref
8. Exemplos e descrições
9. Versionamento e compatibilidade
10. O contrato a serviço de ferramentas
11. As anotações do Springdoc
12. Aplicação no ServiceHub
13. Construindo com IA
14. Glossário
15. Exercícios
16. Para estudar

1. Objetivos da semana

↑ conteúdo

Ao final desta semana você deve ser capaz de:

1. explicar o que um contrato de API fixa e comparar as abordagens design-first e code-first;
2. ler e escrever um documento OpenAPI com `info`, `servers`, `paths`, `components`, `schemas` e `securitySchemes`, usando JSON Schema e `$ref`;
3. classificar uma mudança de contrato como compatível ou como quebra, e decidir quando ela exige nova versão;
4. configurar o Springdoc 3 e documentar controllers e DTOs com `@Tag`, `@Operation`, `@ApiResponse`, `@Parameter` e `@Schema`;
5. documentar cada status real de uma rota, com os erros em ProblemDetail, o esquema `bearerAuth` e os grupos de documentação.

2. O que é um contrato de API

↑ conteúdo

Um **contrato de API** é a descrição precisa de tudo que o cliente pode pedir e de tudo que o servidor promete devolver. Quem consome a API programa contra o contrato, e não contra o código interno. A Semana 01 apresentou essa separação entre interface e implementação; agora a interface ganha forma de documento.

O que o contrato fixa

Rotas e métodos HTTP; parâmetros e corpos de entrada; formato das respostas; códigos de status e formato dos erros; como se autenticar.

Por que escrever

Front-end e back-end trabalham em paralelo; quem consome não precisa adivinhar; ferramentas leem o documento e geram testes, clientes e telas de teste.

Um exemplo fora do ServiceHub: o aplicativo de um banco mostra o saldo porque o contrato diz que `GET /contas/{id}/saldo` devolve um número no campo `valor`, e 404 quando a conta não existe. Se o servidor renomear o campo para `saldo`, o aplicativo quebra, mesmo sem nenhum erro de compilação. Contrato é **promessa**: quem muda o que foi prometido quebra quem confiou nele.

CONFUSÃO COMUM

Achar que documentação de API é só para gente ler. Hoje o documento também é lido por máquinas: geradores de código, simuladores e testes automatizados dependem de que ele esteja certo.

3. Design-first e code-first

↑ conteúdo

Há duas formas de chegar ao documento. A diferença está em quem nasce primeiro e, por isso, em quem é a fonte da verdade.

Aspecto	Design-first	Code-first
Ordem	Escreve o documento OpenAPI, depois implementa.	Implementa, e uma biblioteca gera o documento.
Fonte da verdade	O arquivo YAML ou JSON.	O código anotado.
Vantagem	Consumidores revisam e começam antes de o código existir.	O documento acompanha o código a cada mudança.
Risco	O código divergir do documento sem ninguém notar.	Contrato feito às pressas, refletindo detalhes internos.
Quando usar	API pública, vários times consumindo.	Time único, contrato estável, boa cobertura de testes.

Não existe vencedor absoluto: é uma decisão de arquitetura, boa candidata a um registro de decisão (ADR). No ServiceHub as duas se encontram. O **Mapa de Construção** define o contrato antes do código (design-first), e o Springdoc gera o documento a partir do código (code-first). Testes provam que os dois batem.

> **ARMADILHA**
> Dizer "usamos code-first" quando, na verdade, não houve contrato nenhum antes do código, só código. Code-first ainda exige decidir o contrato; a diferença é onde ele é escrito.

4. OpenAPI, Swagger e Springdoc

↑ conteúdo

Os nomes se misturam em tutoriais. Cada um designa uma coisa diferente:

Nome	O que é	No ServiceHub
OpenAPI Specification	Padrão aberto, em JSON ou YAML, que descreve APIs HTTP. É mantido pela OpenAPI Initiative, ligada à Linux Foundation.	O documento em `/api-docs/v1`
Swagger UI	Página web que lê a especificação, lista as rotas e envia requisições pelo botão *Try it out*.	Aberta em `/docs`
Springdoc OpenAPI	Biblioteca que gera a especificação a partir dos controllers Spring.	`springdoc-openapi-starter-webmvc-ui` 3.0.x

Nome	O que é	No ServiceHub
Springfox	Biblioteca antiga com o mesmo objetivo, sem manutenção.	Não use.

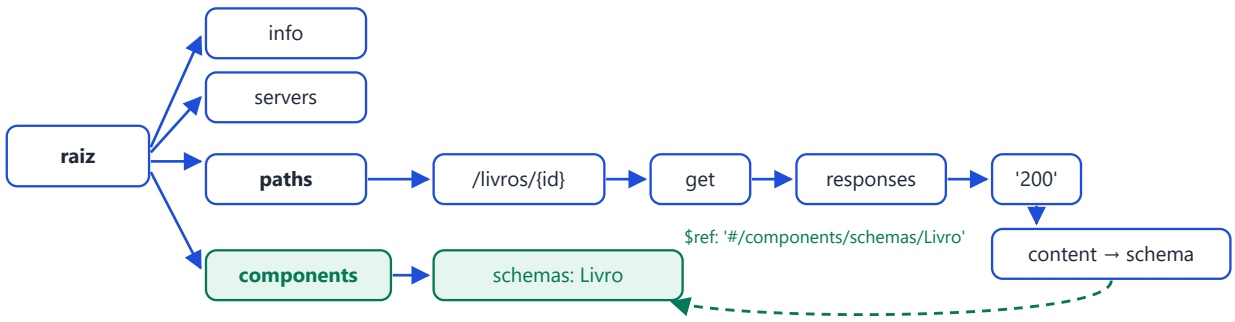
"Swagger" era o nome antigo da especificação. Quando ela foi doada à OpenAPI Initiative, passou a se chamar OpenAPI, e "Swagger" ficou como nome das ferramentas (Swagger UI, Swagger Editor).

5. A estrutura de um documento

↑ conteúdo

Um documento OpenAPI é um único arquivo JSON ou YAML. A raiz declara, no campo `openapi`, a versão da especificação usada (por exemplo, `3.0.3` ou `3.1.0`). Abaixo dela ficam as partes principais:

Parte	Para que serve
`info`	Título, descrição, versão do contrato e contato.
`servers`	Endereços base onde a API roda: local, homologação, produção.
`paths`	Cada rota, como `/livros/{id}`. É o coração do documento.
operations	Dentro de cada rota, um objeto por método (`get`, `post`...), com `parameters`, `requestBody` e `responses`.
`components`	Peças reutilizáveis: `schemas`, `responses`, `parameters`, `examples` e `securitySchemes`.
`security`	Campo opcional na raiz que aplica um esquema de segurança a todas as operações.



Caminho de leitura de um documento OpenAPI: raiz → paths → rota → método → responses → status → content → schema, que aponta para components.

Um exemplo genérico em YAML

Uma biblioteca, fora do ServiceHub, para separar o padrão da ferramenta. Em YAML, a indentação aninha os campos e o hífen abre um item de lista; JSON e YAML são equivalentes.

```yaml
openapi: 3.0.3
info:
  title: Biblioteca API
  version: 1.2.0
servers:
  - url: https://api.biblioteca.exemplo/v1
paths:
  /livros/{id}:
    get:
      summary: Busca um livro pelo id
      parameters:
        - name: id
          in: path
          required: true
          schema: { type: integer, format: int64 }
components:
  schemas:
    Livro:
      type: object
      required: [titulo, isbn]
      properties:
        id:       { type: integer, format: int64 }
        titulo:   { type: string, maxLength: 200 }
        situacao: { type: string, enum: [DISPONIVEL, EMPRESTADO] }
  securitySchemes:
    bearerAuth: { type: http, scheme: bearer, bearerFormat: JWT }
```

- `openapi` é a versão do *padrão*; `info.version` é a versão do *seu contrato*. A diferença costuma cair em prova.
- `{id}` na rota exige um parâmetro `in: path` obrigatório. Outros lugares de parâmetro: `query`, `header` e `cookie`.
- `securitySchemes` só *declara* o mecanismo de autenticação; cada operação (ou o campo raiz `security`) diz se o exige. Declarar não protege nada: quem protege é o servidor, no ServiceHub o Spring Security.

6. JSON Schema: descrevendo dados

↑ conteúdo

Os `schemas` do OpenAPI usam o vocabulário do **JSON Schema**: palavras-chave que dizem que tipo um valor tem e que regras precisa cumprir. A versão 3.1 do OpenAPI se alinhou por completo ao JSON Schema; a 3.0 usa um subconjunto dele. As palavras básicas são as mesmas nas duas:

Palavra-chave	Significado	De onde vem no Java (Springdoc)
`type`	`string`, `integer`, `number`, `boolean`, `array`, `object`	Tipo do campo no DTO

Palavra-chave	Significado	De onde vem no Java (Springdoc)
`required`	Campos que não podem faltar	`@NotNull`, `@NotBlank`
`format`	Refina o tipo: `int64`, `date-time`, `email`, `uuid`	`Long`, `LocalDateTime`, `UUID`
`enum`	Conjunto fechado de valores aceitos	Um `enum` Java, como `ServiceCategory`
`minLength`, `maxLength`	Limites de tamanho de texto	`@Size(min, max)`
`minimum`, `maximum`	Limites numéricos	`@Min`, `@Max`, `@DecimalMin`

O ponto importante: a Bean Validation da Semana 04 não é só validação em tempo de execução. O Springdoc lê essas anotações e as escreve no contrato. Um `@Size(max = 150)` no título vira `maxLength: 150` no schema, sem você escrever de novo.

PARA LEMBRAR

`format` é uma dica para quem consome. Quem garante a regra continua sendo a validação no servidor: um cliente pode ignorar o documento e mandar qualquer coisa.

7. Reuso com $ref

↑ conteúdo

`$ref` é uma **referência**: no lugar do conteúdo, um caminho até onde ele está definido. O caminho segue o padrão JSON Pointer: `#/` é a raiz do próprio documento, e cada `/` desce um nível.

```
paths:
  /livros/{id}:
    get:
      responses:
        '200':
          description: Livro encontrado
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Livro'
        '404':
          $ref: '#/components/responses/NaoEncontrado'
```

Mudou `Livro`? Todas as rotas que apontam para ele mudam juntas. É o princípio de não repetir (DRY) aplicado ao contrato. O `$ref` também pode apontar para outro arquivo, o que ajuda em contratos grandes escritos à mão. No code-first, o Springdoc gera os `$ref` sozinho: cada DTO aparece uma vez em `components/schemas`, e as rotas só apontam para ele.

8. Exemplos e descrições

↑ conteúdo

Um contrato que só tem tipos responde "o quê", mas não "por quê" nem "quando". Três elementos fazem o contrato se explicar:

- **`summary` e `description`:** `summary` é a linha curta da lista de rotas; `description` explica regras, quem pode chamar e efeitos colaterais.
- **`example`:** um valor realista por campo. Ferramentas usam o exemplo para pré-preencher o *Try it out*, montar testes e simular respostas.
- **Erros descritos:** cada status diz *quando* ocorre. "409 se o e-mail já existe", e não só "Conflict".

Campo	Descrição fraca	Descrição útil
`isbn`	`string` · "ISBN"	13 dígitos, sem hífens · exemplo `9788550808161`
`price`	`number` · "Preço"	Preço em reais, maior que zero, duas casas · exemplo `150.00`

ERRO COMUM EM CÓDIGO GERADO POR IA

`example = "string"`, descrições que só repetem o nome do campo e respostas de erro sem dizer quando acontecem. Regra prática: uma boa descrição responde o que o tipo sozinho não responde.

9. Versionamento e compatibilidade

↑ conteúdo

Há duas versões diferentes num contrato, e confundi-las é comum.

info.version: 1.2.0

Versão do *documento*. Muda a cada alteração do contrato e costuma seguir o versionamento semântico, MAJOR.MINOR.PATCH: MAJOR quebra clientes, MINOR acrescenta sem quebrar, PATCH corrige texto ou exemplo.

/api/v1/...

Versão *na URL*. Muda só quando há quebra. A v1 continua no ar enquanto os clientes migram para a v2. Alternativas: cabeçalho ou tipo de mídia; a URL é a mais simples de ver e de testar.

Por isso todas as rotas do ServiceHub começam em `/api/v1`: a porta para uma v2 fica aberta desde o primeiro dia. Mas criar `/api/v2` para qualquer ajuste multiplica código a manter. A pergunta certa é: *a mudança quebra quem já usa?*

Mudança no contrato	Efeito em quem já consome
Nova rota ou novo parâmetro opcional	Compatível: ninguém é obrigado a usar.
Novo campo na resposta	Compatível, se o cliente ignora campos desconhecidos.
Remover ou renomear campo da resposta	Quebra: o cliente lê um campo que sumiu.
Campo opcional da entrada vira obrigatório	Quebra: requisições antigas passam a dar 400.
Trocar o tipo de um campo (número para texto)	Quebra: a leitura falha no cliente.
Trocar o status de sucesso (200 para 201)	Quebra clientes que testam o código exato.

A regra que resume a tabela é uma leitura do **princípio da robustez**: na entrada, seja tolerante; na saída, nunca retire o que já prometeu. Um caso sutil: acrescentar um valor novo a um `enum` da *resposta* pode quebrar clientes que tratam só os valores conhecidos.

10. O contrato a serviço de ferramentas

↑ conteúdo

Um documento legível por máquina gera trabalho pronto. Três usos aparecem em qualquer equipe que leva o contrato a sério (aqui, só o conceito; não vamos gerar clientes na disciplina):

- **Cliente gerado:** um gerador, como o OpenAPI Generator, lê o documento e produz classes e métodos tipados em TypeScript, Java, Kotlin e outras linguagens.
- **Servidor simulado (mock):** responde com os exemplos do contrato. O front-end trabalha antes de o back-end existir.
- **Teste de contrato:** confere se a implementação responde como o contrato promete, com os mesmos status, campos e tipos, e falha antes que o cliente descubra. Na variante guiada pelo consumidor, cada cliente declara o que espera e o servidor prova que atende.

Ferramentas de apoio consomem o mesmo arquivo: o **Swagger UI** lista as rotas e envia requisições; o **Swagger Editor** valida o YAML com prévia ao lado, útil no design-first; o **Postman**, cliente HTTP com coleções, variáveis de ambiente e testes, importa um documento OpenAPI e cria a coleção. É a vantagem de um padrão aberto.

11. As anotações do Springdoc

↑ conteúdo

Só com a dependência no `pom.xml`, o Springdoc já documenta todas as rotas, mas mal: sem descrições, com status adivinhados e sem exemplos. As anotações transformam esse rascunho em contrato. Cada uma responde a uma pergunta de quem consome:

Pergunta	Anotação	Onde
Em que grupo está?	`@Tag(name, description)`	Na classe do controller, uma por recurso.

Pergunta	Anotação	Onde
O que a rota faz?	`@Operation(summary, description)`	No método.
O que pode voltar?	`@ApiResponses` / `@ApiResponse`	No método, um por status.
O que eu envio?	`@Parameter`, `@ParameterObject`, `@Schema`	Nos parâmetros e nos campos dos DTOs.

Um controller genérico de livros, documentado (resumido):

```
@RestController
@RequestMapping("/livros")
@Tag(name = "Livros", description = "Acervo da biblioteca")
public class LivroController {

  @Operation(summary = "Buscar livro por id",
      description = "Público. Livro retirado do acervo responde 404.")
  @ApiResponses({
    @ApiResponse(responseCode = "200", description = "Livro encontrado"),
    @ApiResponse(responseCode = "404", description = "Livro não existe",
      content = @Content(mediaType = "application/problem+json",
        schema = @Schema(implementation = ProblemDetail.class),
        examples = @ExampleObject(name = "nao-encontrado", value = """
          {"type": "urn:biblioteca:problem:not-found",
           "title": "Recurso não encontrado", "status": 404,
           "detail": "Livro não encontrado: 99", "instance": "/livros/99"}"""))
  })
  @GetMapping("/{id}")
  public LivroResponse buscar(
      @Parameter(description = "Id do livro", example = "42") @PathVariable Long id) { ... }
}
```

- **`summary`** é verbo e objeto, curto; **`description`** diz quem pode chamar e o que a rota decide sozinha.
- Cada **`@ApiResponse`** de erro aponta para `ProblemDetail`, com o tipo de mídia `application/problem+json`. O **`@ExampleObject`** mostra no Swagger UI um corpo de erro realista.
- **`@ParameterObject`**, usado num `Pageable`, abre o objeto em `page`, `size` e `sort`, um campo cada no formulário.
- **`@Schema(description, example)`** num campo de record vira descrição e exemplo do schema; o exemplo pré-preenche o corpo do *Try it out*.
- **`@SecurityRequirements`** vazio numa operação diz "esta rota é pública", tirando o cadeado que um requisito global de segurança colocaria.

DOCUMENTE O QUE EXISTE, NEM MAIS NEM MENOS

Dois erros opostos. Documentar 400, 401, 403, 404, 409 e 500 em toda rota "por garantia" mente sobre o contrato. Inventar uma classe `ErrorResponse` que não existe no projeto também: o erro real é `ProblemDetail`. Cada status documentado deve corresponder a uma condição que o código produz ou que o contrato do Mapa exige.

12. Aplicação no ServiceHub

↑ conteúdo

Os trechos abaixo são do código real do projeto. Cortes estão marcados como "resumido".

Dependência e propriedades

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>3.0.3</version>
</dependency>

# application.properties
springdoc.api-docs.path=/api-docs
springdoc.swagger-ui.path=/docs
springdoc.swagger-ui.operations-sorter=method
springdoc.swagger-ui.tags-sorter=alpha
springdoc.swagger-ui.doc-expansion=none
```

A linha 3.x do Springdoc é a que acompanha o Spring Boot 4; a 2.x, comum em tutoriais, não funciona com ele. Os caminhos padrão seriam `/v3/api-docs` e `/swagger-ui.html`; o ServiceHub usa `/docs` e `/api-docs`, como o Mapa pede.

O bean OpenAPI: cabeçalho e segurança

`config/OpenApiConfig.java` (resumido; a descrição completa e a licença foram cortadas):

```
@Bean
public OpenAPI servicehubOpenAPI(ObjectProvider<BuildProperties> buildProperties) {
    BuildProperties props = buildProperties.getIfAvailable();
    return new OpenAPI()
        .info(new Info().title("ServiceHub API")
            .version(props != null ? props.getVersion() : "dev"))
        .servers(List.of(
            new Server().url("http://localhost:8080").description("Execução local"),
            new Server().url("http://localhost:8090").description("Docker Compose")))
        .components(new Components().addSecuritySchemes(BEARER, new SecurityScheme()
            .type(SecurityScheme.Type.HTTP).scheme("bearer").bearerFormat("JWT")))
```

```
        .addSecurityItem(new SecurityRequirement().addList(BEARER));
}
```

O `addSecurityItem` global marca todas as rotas como protegidas; as públicas (cadastro, login, catálogo) declaram `@SecurityRequirements` vazio. Os `servers` viram o seletor de ambiente do *Try it out*. Um segundo bean, `securityResponsesCustomizer` (um `OpenApiCustomizer`), percorre as operações e acrescenta 401 e 403, com schema `ProblemDetail`, a toda rota que não é pública. Assim esses dois status não precisam ser repetidos em cada controller.

Grupos v1 e public

Dois beans `GroupedOpenApi` recortam a documentação. O grupo `v1` (`.group("v1").pathsToMatch("/api/v1/**")`) traz a API inteira; o grupo `public` reúne o que um visitante sem conta usa: cadastro, login, catálogo, avaliações e `/info`, excluindo `/api/v1/services/*/status`. Os dois recebem o `securityResponsesCustomizer`. Cada grupo tem a sua especificação (`/api-docs/v1` e `/api-docs/public`), e um seletor no topo do `/docs` troca entre eles.

Um controller documentado

`controller/ServiceOfferingController.java` (resumido; `PROBLEM` vale `"application/problem+json"`):

```
@Tag(name = "Serviços",
     description = "Catálogo de serviços oferecidos pelos prestadores da plataforma")
public class ServiceOfferingController {

    @Operation(summary = "Cadastrar serviço",
        description = "Apenas PROVIDER. O prestador é o usuário do token "
            + "(providerId não é aceito no corpo). Responde com o serviço "
            + "criado e o header Location.")
    @ApiResponses({
        @ApiResponse(responseCode = "201", description = "Serviço criado"),
        @ApiResponse(responseCode = "400",
            description = "Campos inválidos, categoria inexistente ou JSON malformado",
            content = @Content(mediaType = PROBLEM,
                schema = @Schema(implementation = ProblemDetail.class))),
        @ApiResponse(responseCode = "403",
            description = "Apenas PROVIDER publica serviços",
            content = @Content(mediaType = PROBLEM,
                schema = @Schema(implementation = ProblemDetail.class)))
    })
    @PostMapping
    public ResponseEntity<ServiceOfferingResponseDTO> create(
            @Valid @RequestBody ServiceOfferingDTO dto) { ... }
```

A `description` conta uma regra de segurança (o prestador vem do token): quem consome precisa saber disso para montar o corpo certo. Na listagem, cada filtro tem `@Parameter(description,`

`example)`, o `Pageable` recebe `@ParameterObject`, e a `@Operation` diz em texto o tamanho padrão (20), o máximo (100) e os campos aceitos em `sort`, porque o teto não aparece sozinho na especificação.

Um DTO com @Schema

```
@Schema(description = "Abertura de ordem de serviço. "
        + "Cliente, status e preço são definidos pela API.")
public record ServiceRequestCreateDTO(
        @Schema(description = "ID do serviço contratado", example = "12")
        @NotNull(message = "O serviço é obrigatório")
        @Positive(message = "O id do serviço deve ser positivo")
        Long serviceId,

        @Schema(description = "Data e hora desejadas (horário de Brasília)",
                example = "2026-11-10T09:00:00")
        @Future(message = "A data agendada deve estar no futuro")
        LocalDateTime scheduledAt,
        String notes   // @Schema e @Size(max = 1000) omitidos (resumido)
) { }
```

A descrição da classe avisa o que o cliente *não* envia: é a defesa contra mass assignment da Semana 04, agora escrita no contrato.

O teste de contrato do projeto

`OpenApiIntegrationTest` (resumido) busca a especificação real e confere o que o contrato promete:

```
@Test
@DisplayName("deve documentar o esquema bearerAuth (botão Authorize no Swagger)")
void deveDocumentarEsquemaBearer() {
    String body = rest.getForEntity("/api-docs/v1", String.class).getBody();
    assertThat(body).contains("\"bearerAuth\"")
            .contains("\"scheme\":\"bearer\"").contains("\"bearerFormat\":\"JWT\"");
}
```

Os outros casos da mesma classe conferem que `/docs` abre, que o grupo `public` não expõe `/api/v1/users` e que a especificação menciona `ProblemDetail`.

13. Construindo com IA

↑ conteúdo

Documentação é onde a IA mais "inventa": ela lista status plausíveis em vez dos reais. Por isso a especificação vem do Mapa e do código, nunca do assistente.

- **Especificar:** escreva a tabela de status por rota a partir da seção 6.3 do Mapa.
- **Gerar:** peça um plano que ligue cada status a uma condição do código; só depois as anotações.
- **Revisar:** cada status bate com uma exceção do código ou com o Mapa? Nenhum 500?
- **Provar:** um corpo inválido no `/docs` devolve o 400 documentado.

Prompt fraco e prompt bom

Fraco

```
adiciona swagger no projeto e documenta
tudo
```

Vêm Springfox, 500 em toda rota e uma `ErrorResponse` inventada.

Bom

Fixa versão e caminhos, entrega a tabela de status, exige ProblemDetail com exemplo, pede plano e um aceite verificável. Está logo abaixo.

O prompt desta semana

Ponto de partida: o projeto da Semana 04, com o CRUD de serviços e o `GlobalExceptionHandler`.

```
Contexto: API ServiceHub, Java 21, Spring Boot 4.1, pacote
com.servicehub. Já existem ServiceOfferingController (CRUD em
/api/v1/services), DTOs em record e GlobalExceptionHandler com
ProblemDetail (RFC 9457), type urn:servicehub:problem:<tipo>.
O contrato é a seção 6.3 do Mapa (anexo).

Objetivo: documentar o ServiceOfferingController no OpenAPI.

Status por rota (seção 6.3 do Mapa):
- POST   /services        201 · 400 · 401 · 403
- GET    /services        200 · 400 (filtro ou sort inválido)
- GET    /services/{id}   200 · 404
- PUT    /services/{id}   200 · 400 · 401 · 403 · 404 · 409
- DELETE /services/{id}   204 · 401 · 403 · 404 · 409

Restrições:
- springdoc-openapi-starter-webmvc-ui 3.0.x. NÃO use Springfox nem
  springdoc 2.x. Caminhos: springdoc.swagger-ui.path=/docs e
  springdoc.api-docs.path=/api-docs.
- @Tag na classe; @Operation com summary e description em cada rota.
- Cada erro: @ApiResponse com @Content(mediaType =
  "application/problem+json", schema = @Schema(implementation =
```

```
  ProblemDetail.class)) e um @ExampleObject com o JSON do Mapa
  (type, title, status, detail, instance).
- NUNCA documente 500. NÃO crie classe ErrorResponse.
- 401 e 403 de token: declare o esquema bearerAuth no bean OpenAPI
  e um OpenApiCustomizer que os acrescenta às rotas protegidas. As
  rotas GET públicas recebem @SecurityRequirements vazio.
- Listagem: @Parameter com exemplo em cada filtro e @ParameterObject
  no Pageable; a description informa padrão 20, máximo 100 e os
  campos aceitos em sort.
- @Schema com exemplo do domínio nos DTOs (nunca "string").
- Não altere o comportamento de nenhuma rota.

Antes do código, mostre um plano: uma tabela com rota, status,
condição que o produz (exceção ou regra) e type. Marque como
"pendente" o status que o Mapa exige e o código ainda não produz
(ex.: 401/403 antes da Semana 08; 409 antes das ordens).

Critério de aceite: o Swagger em /docs mostra, em cada rota, cada
status da tabela com o schema ProblemDetail e o exemplo; nenhuma
rota mostra 500; /api-docs/v1 contém "bearerAuth".
```

O que a IA costuma errar nesta etapa

- Springfox ou springdoc 2.x, que não funcionam com o Boot 4; caminhos padrão `/v3/api-docs` e `/swagger-ui.html`.
- Inventar `ErrorResponse` em vez de usar `ProblemDetail`.
- Listar "400 a 500 em toda rota por garantia", inclusive status que a rota não devolve.
- Esquecer `@SecurityRequirements` nas rotas públicas, que ficam com cadeado no `/docs`.

Checklist de revisão

☐ Springdoc 3.0.x no `pom.xml`? `/docs` e `/api-docs` nas propriedades?

☐ Cada status anotado aparece na tabela do plano, ligado a uma exceção do `GlobalExceptionHandler` ou a uma regra do Mapa?

☐ Todo erro usa `application/problem+json` e `ProblemDetail`, com exemplo cujo `type` existe em `ProblemTypes`? Nenhuma rota documenta 500?

☐ O `git diff` mostra só anotações e configuração?

Prova

```
mvnw.cmd spring-boot:run
curl -s http://localhost:8080/api-docs/v1
```

Abra `http://localhost:8080/docs`: `POST /api/v1/services` mostra 201, 400, 401 e 403, cada erro com schema e exemplo; envie `"price": 0` pelo *Try it out* e compare com o exemplo. Feche com um teste que lê `/api-docs/v1` e confere `bearerAuth` e `ProblemDetail`.

14. Glossário

↑ conteúdo

Contrato de API	Descrição precisa do que o cliente pode pedir e do que o servidor promete devolver.
OpenAPI	Padrão aberto, em JSON ou YAML, para descrever APIs HTTP.
Swagger UI	Página que lê um documento OpenAPI, lista as rotas e permite testá-las.
Springdoc	Biblioteca que gera o documento OpenAPI a partir de controllers Spring.
Design-first	Abordagem em que o documento é escrito antes do código.
Code-first	Abordagem em que o documento é gerado a partir do código anotado.
Path / operation	Uma rota do documento e, dentro dela, o objeto de cada método HTTP.
Components	Seção de peças reutilizáveis: schemas, respostas, parâmetros, exemplos, segurança.
Schema	Descrição da forma de um dado, com o vocabulário do JSON Schema.
$ref	Referência a uma definição feita em outro ponto do documento.
securityScheme	Declaração de um mecanismo de autenticação, como bearer JWT.
Versionamento semântico	Esquema MAJOR.MINOR.PATCH: quebra, acréscimo compatível, correção.
Mudança que quebra	Alteração que faz um cliente existente parar de funcionar.
Teste de contrato	Teste que confere se a implementação responde como o contrato promete.

15. Exercícios

↑ conteúdo

1. Com suas palavras, o que um contrato de API fixa? Por que renomear um campo da resposta quebra o cliente mesmo sem erro de compilação?
2. Uma empresa vai abrir uma API para dez parceiros externos, que precisam começar a integração antes de o back-end ficar pronto. Design-first ou code-first? Justifique e diga qual é o principal risco da abordagem escolhida.
3. Explique a diferença entre OpenAPI, Swagger UI, Springdoc e Springfox.
4. Num documento, a raiz tem `openapi: 3.0.3` e `info.version: 2.4.1`. O que cada número indica?
5. Escreva em YAML a operação `GET /autores/{id}` de uma API de biblioteca, com o parâmetro de caminho, a resposta 200 apontando por `$ref` para o schema `Autor` e a resposta 404.
6. Que palavras-chave de JSON Schema o Springdoc deve gerar para um campo `@NotBlank @Size(min = 3, max = 150) String title`? E para `@DecimalMin("0.01") BigDecimal price`?

7. Classifique como compatível ou quebra, justificando: (a) acrescentar o campo `rating` na resposta de serviço; (b) renomear `price` para `valor`; (c) tornar `scheduledAt` obrigatório na abertura de ordem; (d) criar `GET /api/v1/categories`.

8. Analise a anotação: `@ApiResponse(responseCode = "500", description = "Erro", content = @Content(schema = @Schema(implementation = ErrorResponse.class)))` em `GET /api/v1/services/{id}`. Aponte dois problemas em relação às regras do ServiceHub e reescreva a documentação de erro correta para essa rota.

9. Para que serve `@SecurityRequirements` vazio numa operação? O que acontece no `/docs` se ele faltar em `POST /api/v1/auth/login`?

10. Diferencie cliente gerado, servidor simulado e teste de contrato. Qual dos três o ServiceHub já tem, e onde?

11. Escreva um prompt, no formato da seção 13, para documentar o `AuthController` (`/auth/register`, `/auth/login`, `/auth/me`) com os status da seção 6.1 do Mapa. Inclua o critério de aceite.

16. Para estudar

↑ conteúdo

Leituras disponíveis na Minha Biblioteca, pelo acesso institucional:

- RODRIGUES, T. N. et al. *Integração de aplicações*. Grupo A, 2020. Capítulos "Conceito e uso de web service e APIs" e "Software de apoio" (Postman).
- FOWLER, M. *Padrões de arquitetura de aplicações corporativas*. Bookman, 2006. Capítulo 15, Data Transfer Object: o DTO é exatamente o que vira schema no contrato.

Referências abertas:

- OpenAPI Specification: `spec.openapis.org`. Leia as seções Paths, Components e Schema Object.
- Documentação oficial do Springdoc: `springdoc.org`.
- IETF. RFC 9457: *Problem Details for HTTP APIs*. 2023 (o formato de erro documentado nesta semana).